BHbot – RAG hybride pour l'assurance

Assistant client combinant recherche sémantique (Qdrant), graphe de connaissances (Neo4j) et LLM

Mariem Omrani · Hackathon BH Assurance, septembre 2025 · Version révisée, octobre 2026 · github.com/maryamom/rag-hybrid-assurance

> La solution répond aux questions des clients à partir de documents d'assurance scannés (français et arabe) et du catalogue de produits, donne à un client authentifié l'accès à ses propres contrats et devis, et calcule des devis d'assurance auto. Les données publiques et les données des clients sont strictement séparées : la recherche n'a jamais accès aux secondes.

1. Vision de la solution

1.1 Le besoin

Un assureur veut un assistant, intégré à son site vitrine, capable de :

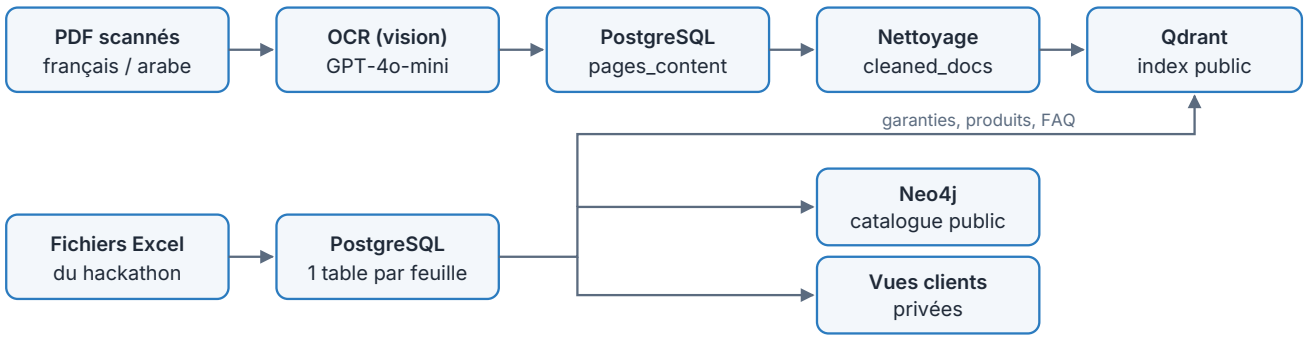
- **comprendre** des questions en langage naturel sur les garanties, les produits et les démarches ;
- **donner accès** à ses contrats et devis à un client, après vérification de son identité ;
- **calculer** un devis auto personnalisé en temps réel.

1.2 Quatre composantes

Composante	Rôle
Recherche sémantique (Qdrant)	Retrouve les passages de documents dont le sens est proche de la question, grâce à des embeddings multilingues.
Recherche relationnelle (Neo4j)	Interroge un graphe produits – garanties – branches – profils clients, pour les questions de relations et de comparaison.
Génération (LLM via OpenRouter)	Rédige la réponse à partir des seuls extraits retrouvés, en citant ses sources. DeepSeek v3.1 par défaut, modèle configurable.
Orchestration (FastAPI, LangChain)	Oriente chaque question vers le bon traitement, lance les deux recherches en parallèle, fusionne leurs résultats.

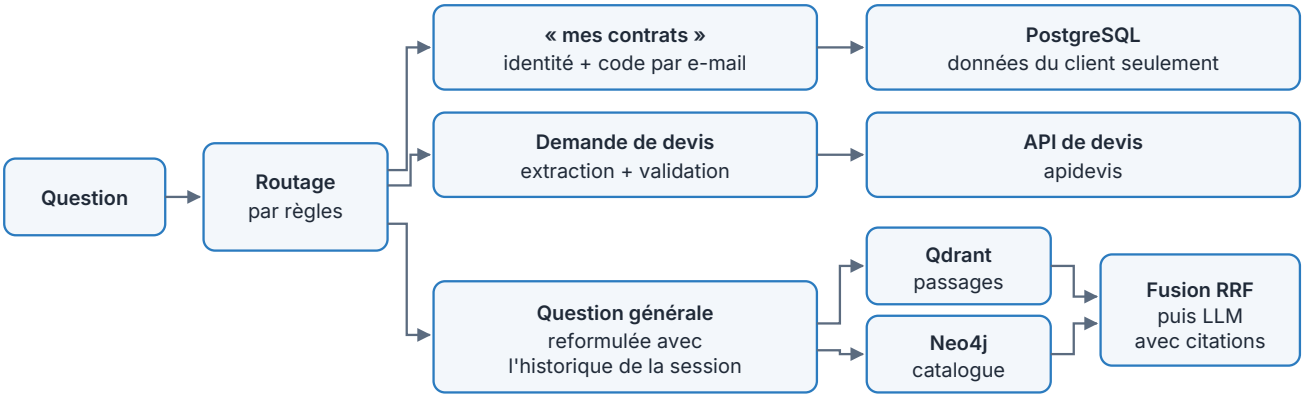
2. Architecture

2.1 Préparation des données (hors ligne)



Les documents et les tables publiques alimentent Qdrant et Neo4j ; les données clients restent dans PostgreSQL.

2.2 Traitement d'une question (en ligne)



Le routage a lieu avant tout appel au LLM ; Qdrant et Neo4j sont interrogés en parallèle.

3. Étapes techniques

3.1 Préparation des données

OCR par modèle de vision

La majorité des documents sont des PDF scannés, en français et en arabe, souvent de qualité médiocre : un OCR classique donnait un texte inexploitable. Chaque page est donc rendue en image haute résolution (PyMuPDF, zoom ×2) puis transcrite fidèlement, dans sa langue d'origine, par un modèle de vision appelé via OpenRouter (`OCR_MODEL`, GPT-4o-mini par défaut). Le texte de chaque page est stocké dans PostgreSQL (`documents`, `pages_content`).

Nettoyage

Les pages d'un document sont réunies, les en-têtes, pieds de page et numéros de page répétés sont supprimés, les espaces sont normalisés. La branche d'assurance est déduite du dossier du PDF. Le résultat (`cleaned_docs`) peut être exporté en JSONL pour reconstruire les index sans refaire l'OCR.

Index vectoriel (Qdrant)

Les documents sont découpés en extraits d'environ 1 000 caractères (avec chevauchement), vectorisés par un modèle multilingue (`paraphrase-multilingual-MiniLM-L12-v2`) et indexés avec leur type. Seules les données publiques sont indexées : documents, descriptions de garanties, correspondance produits – profils, questions-réponses et dictionnaire des colonnes.

Graphe de connaissances (Neo4j)

Le graphe relie **Branche** → **SousBranche** → **Produit** → **Garantie**, ainsi que les **ProfilCible** de chaque produit et les **QuestionReponse** d'exemple. Il est construit à partir des seuls noms distincts de produits, branches et garanties : aucune personne, aucun numéro de contrat et aucun montant n'y figure. Chaque nœud porte une description textuelle, couverte par un index plein texte avec analyseur français.

3.2 Moteur de recherche hybride

Recherche sémantique

Embedding de la question, puis une recherche de similarité cosinus dans Qdrant, filtrée sur les types publics. Les résultats sont triés par score.

Recherche relationnelle

Une requête sur l'index plein texte de Neo4j, limitée aux étiquettes publiques, avec les nœuds voisins (par exemple les produits qui incluent une garantie).

Les deux recherches sont lancées **en parallèle**, avec un délai maximal chacune : si une base est indisponible, l'autre répond quand même. Leurs scores n'étant pas comparables (cosinus et Lucene), les classements sont fusionnés par **Reciprocal Rank Fusion** :

$$\text{score}(d) = \Sigma_{\text{sources}}\ 1 / (60 + \text{rang}_{\text{source}}(d))$$

Un passage bien classé par les deux sources arrive en tête ; les doublons sont fusionnés.

3.3 Génération de la réponse

- Si la conversation a un historique, la question est d'abord **reformulée** pour être compréhensible seule (« et la franchise ? »).
- Le LLM reçoit les extraits numérotés et répond **uniquement** à partir d'eux, en citant ses sources [1], [2] ; il dit clairement quand l'information n'y figure pas.

- Température 0,1, montants en dinars tunisiens, réponse dans la langue de la question.
- Sans clé d'API, l'assistant affiche les passages les plus pertinents au lieu de générer une réponse.

3.4 Accès sécurisé aux données personnelles

#	Étape
1	Détection de la demande (« mes contrats », « ma prime »…) par des règles, avant tout appel au LLM.
2	Le client donne prénom, nom et téléphone ; la réponse est la même que le client existe ou non, et l'adresse e-mail n'est jamais affichée.
3	Un code à 6 chiffres (générateur cryptographique) est envoyé à l'adresse enregistrée. Il est stocké haché, lié à la session du navigateur, valable 10 minutes, bloqué après 3 essais.
4	Une fois le code validé, les contrats et devis sont lus dans PostgreSQL, filtrés sur l'identifiant du client ; un jeton d'une heure permet les questions de suivi.
5	Les journaux ne contiennent ni question, ni identité, ni CIN ; les données du client ne sont pas réutilisées dans les prompts suivants.

3.5 Devis auto

- Les informations du véhicule sont extraites du message par le LLM (JSON), avec des expressions régulières en secours, puis **validées par Pydantic** (CIN à 8 chiffres, date passée, valeurs plausibles).
- Aucune valeur n'est inventée : l'assistant demande les informations manquantes et les complète au fil de la conversation.
- Le devis est calculé par l'API de tarification du hackathon ; le CIN est masqué dans l'historique.

4. Qualité

- **75 tests automatisés** : routage, extraction, authentification, fusion, sessions, API HTTP, et tests d'intégration sur de vraies bases PostgreSQL et Neo4j, exécutés par GitHub Actions à chaque push.
- **Évaluation** : `eval/evaluate.py` compare recherche vectorielle seule, graphe seul et hybride (taux de réussite @5, MRR, latence médiane et p95) sur des questions rédigées à partir des documents.

5. Limites et pistes

- Sessions et codes sont conservés en mémoire : il faudrait Redis pour plusieurs processus.
- Les tentatives sont limitées par session ; un déploiement public demanderait aussi une limite par adresse IP.
- Le routage par règles est fiable sur les questions du hackathon, mais un classifieur généraliserait mieux.
- Les devis dépendent d'une API externe fournie pour le hackathon.

6. Du hackathon à la version actuelle

La version du hackathon (étiquette Git `hackathon-2025-09`) a été auditée en octobre 2026. Cet audit a conduit à séparer strictement données publiques et données clients dans la recherche, à isoler chaque session, à renforcer le code de vérification, à remplacer la concaténation des résultats par une fusion RRF en parallèle, à passer à des embeddings multilingues, à ajouter les scripts d'ingestion manquants, à découper le code en modules et à ajouter les tests.